ANSIBLE

Complete Training Guide

Automation • Configuration Management • Orchestration

Linux Mint | AlmaLinux | Ad-Hoc Commands | Playbooks | Inventory

Instructor-led Session | Hands-On Labs Included

Session Agenda

01 **What is Ansible?**
Introduction & core concepts

02 **Why & When to Use Ansible**
Use cases and advantages

03 **How Ansible Works**
Architecture & components

04 **Installation & Setup**
Linux Mint & AlmaLinux

05 **Inventory & Configuration**
Hosts, groups, ansible.cfg

06 **Ad-Hoc Commands**
Quick one-liner tasks

07 **Playbooks**
YAML automation scripts

08 **Hands-On Lab**
Practice exercises

What is Ansible?

Ansible is an open-source IT automation tool developed by Red Hat.

It automates software provisioning, configuration management, application deployment, and orchestration — using simple YAML files called Playbooks.

Agentless	Idempotent	YAML-Based	Push-Based
No software needed on managed nodes — uses SSH	Running same task multiple times = same result	Human-readable playbooks, easy to learn	Control node pushes config to managed nodes

🏛️ Originally released in 2012 by Michael DeHaan — acquired by Red Hat (IBM) in 2015

What is Ansible Used For?

⚙️ Configuration Management

Set and enforce system config: packages, files, users, services across 100s of servers

🚀 Application Deployment

Deploy apps consistently to dev, staging, production environments in one command

📦 Provisioning

Set up new servers, cloud instances (AWS, Azure, GCP) from scratch automatically

🔄 Orchestration

Coordinate multi-step workflows: stop → update → test → start across many hosts

🔐 Security & Compliance

Enforce security policies, patch systems, audit configurations at scale

☁️ Cloud Automation

Manage cloud resources, containers (Docker/K8s), and networking devices

When to Use Ansible?

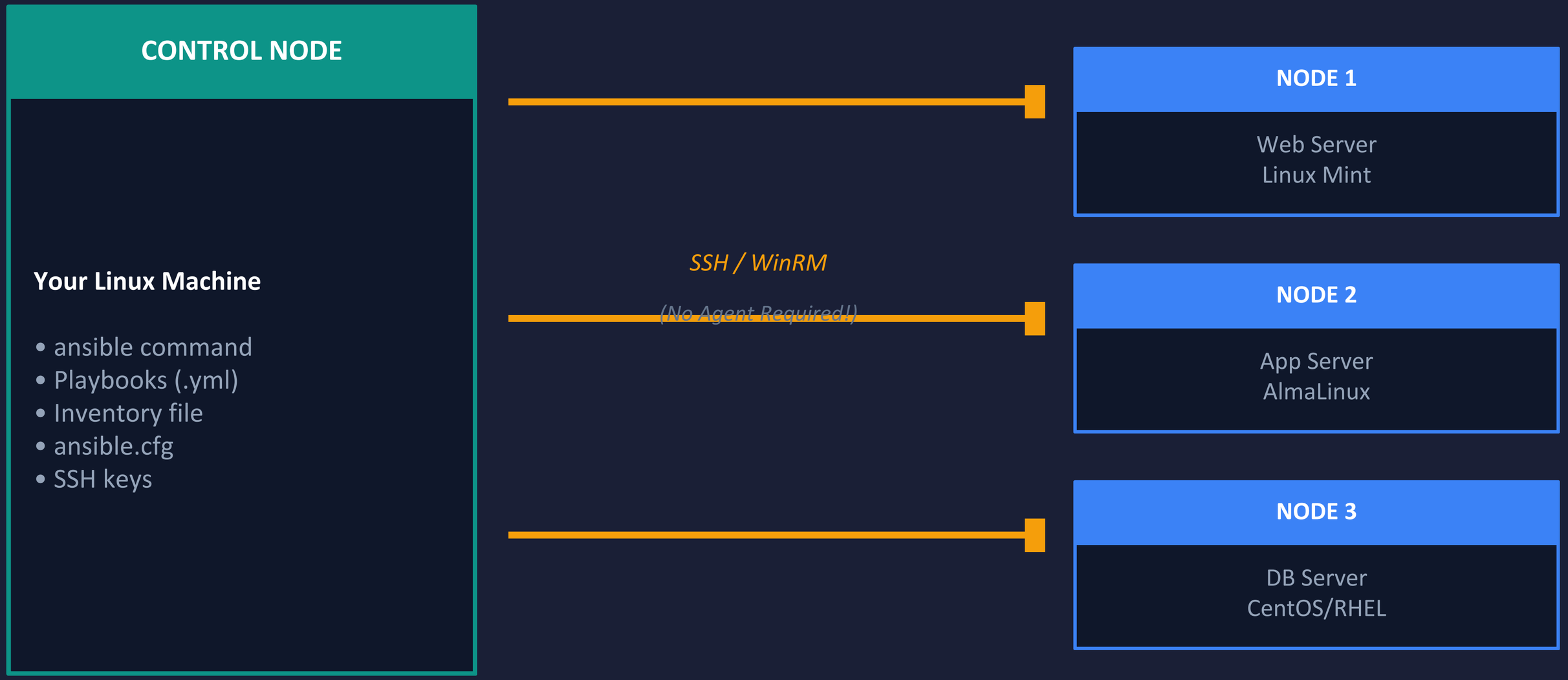
✅ USE Ansible When...

- You manage 10+ servers and do repetitive tasks
- You need consistent environments (dev/staging/prod)
- You want to automate deployments & reduce errors
- Your team uses Linux/Unix-based infrastructure
- You need quick setup without learning complex tools
- You want infrastructure-as-code practices

❌ Consider Alternatives When...

- You need state management → Use Terraform
- Complex orchestration flows → Use Jenkins/Airflow
- Container-native config → Use Helm/K8s
- Windows-heavy environments (limited support)
- Real-time monitoring → Use Nagios/Prometheus
- Single server with simple tasks → Use shell scripts

How Ansible Works — Architecture



💡 Ansible pushes modules via SSH → runs on managed node → returns results → removes modules

Core Components of Ansible

📋 Inventory

List of managed hosts (IP/hostnames). Static file or dynamic.

```
INI or YAML format
[webservers]
192.168.1.10
192.168.1.11
```

📁 Playbook

YAML file defining automation tasks. Heart of Ansible.

```
---
- hosts: webservers
  tasks:
  - name: Install nginx
    yum: name=nginx
```

🧩 Module

Built-in units of work: apt, yum, copy, service, user, etc.

```
2000+ built-in modules
Examples: ping, shell,
copy, file, template
```

👤 Role

Reusable, organized collection of tasks/files/templates.

```
roles/
  webserver/
    tasks/main.yml
    files/
    templates/
```

🔧 Variables

Dynamic values used in playbooks and templates.

```
vars:
  http_port: 80
  app_name: myapp
```

🔐 Vault

Encrypt sensitive data like passwords and API keys.

```
ansible-vault encrypt
secrets.yml
Passphrase protected
```

Installation — Linux Mint (Ubuntu-based)

🐧 Linux Mint / Ubuntu / Debian — APT Package Manager

1	Update Package Index	`sudo apt update && sudo apt upgrade -y`
2	Install Software Properties	`sudo apt install software-properties-common -y`
3	Add Ansible PPA Repository	`sudo add-apt-repository --yes --update ppa:ansible/ansible`
4	Install Ansible	`sudo apt install ansible -y`
5	Verify Installation	`ansible --version`

```
Expected output: ansible [core 2.x.x] | python version = 3.x.x | jinja version = 3.x.x
```

Installation — AlmaLinux (RHEL-based)

AlmaLinux / Rocky Linux / CentOS / RHEL — DNF/YUM Package Manager

1	Update System Packages	`sudo dnf update -y`
2	Install EPEL Repository	`sudo dnf install epel-release -y`
3	Install Ansible	`sudo dnf install ansible -y`
4	Verify Installation	`ansible --version`
5	Install via pip (alternative)	`pip3 install ansible  OR  pip3 install ansible-core`

📌 SSH must be enabled on managed nodes: sudo systemctl enable --now sshd

Inventory File & ansible.cfg

Inventory File /etc/ansible/hosts

```
# Static Inventory Example

[webservers]
192.168.1.10
192.168.1.11  ansible_user=admin

[dbservers]
db01.example.com
db02.example.com

[all:vars]
ansible_python_interpreter=/usr/bin/python3

# INI group of groups
[production:children]
webservers
dbservers
```

ansible.cfg Configuration File

```
# /etc/ansible/ansible.cfg

[defaults]
inventory      = /etc/ansible/hosts
remote_user    = ansible
host_key_checking = False
log_path       = /var/log/ansible.log
forks          = 10

[privilege_escalation]
become         = True
become_method  = sudo
become_user    = root
become_ask_pass = False
```

Ad-Hoc Commands

💡 Ad-Hoc commands run a single Ansible task quickly — without writing a Playbook. Perfect for one-time tasks.

SYNTAX:

```
ansible <pattern> -m <module> -a '<arguments>' [options]
```

Ping all hosts

```
ansible all -m ping
```

Ping a group

```
ansible webservers -m ping
```

Check uptime

```
ansible all -m command -a 'uptime'
```

Free memory

```
ansible all -m command -a 'free -m'
```

Install package

```
ansible webservers -m yum -a "name=nginx state=present" -b
```

Remove package

```
ansible webservers -m apt -a "name=apache2 state=absent" -b
```

Start service

```
ansible all -m service -a 'name=nginx state=started' -b
```

Copy file

```
ansible all -m copy -a "src=/tmp/file.txt dest=/tmp/file.txt"
```

Create user

```
ansible all -m user -a 'name=john state=present' -b
```

Reboot hosts

```
ansible all -m reboot -b
```

Ansible Playbooks — Introduction

What is a Playbook?

A Playbook is a YAML file containing a list of plays. Each play maps a group of hosts to a set of tasks to run. Playbooks are the primary way to automate complex, multi-step IT processes with Ansible.

- ✓ **Written in YAML**
  Human-readable format
- ✓ **Idempotent**
  Safe to run multiple times
- ✓ **Ordered**
  Tasks execute in sequence
- ✓ **Reusable**
  Can import other playbooks/roles

📄 example-playbook.yml

```yaml
---
- name: Setup Web Server
  hosts: webservers
  become: yes
  vars:
    http_port: 80
    pkg_name: nginx

  tasks:
    - name: Install nginx
      yum:
        name: "{{ pkg_name }}"
        state: present

    - name: Start nginx service
      service:
        name: nginx
        state: started
        enabled: yes

    - name: Open firewall port
      firewalld:
        port: "{{ http_port }}/tcp"
        state: enabled
        permanent: yes
```

Playbook Structure — Deep Dive

`---`	**YAML Document Start**	Always begin playbooks with three dashes
`- name:`	**Play Name**	Human-friendly description of what this play does
`hosts:`	**Target Hosts**	Which inventory group/host to run against: all, webservers, 192.168.1.1
`become:`	**Privilege Escalation**	become: yes → run as root via sudo
`vars:`	**Variables**	Key-value pairs used throughout the playbook
`tasks:`	**Task List**	List of modules to execute in order on the hosts
`handlers:`	**Handlers**	Tasks triggered by notify: (e.g. restart nginx when config changes)
`roles:`	**Roles**	Include reusable role directories for clean organization

Essential Ansible Modules

Package Management

`apt`	Install/remove packages (Debian/Ubuntu/Mint)
`yum / dnf`	Install/remove packages (RHEL/AlmaLinux/CentOS)
`pip`	Manage Python packages

File Management

`copy`	Copy files to remote nodes
`template`	Deploy Jinja2 templates
`file`	Manage file/dir permissions and ownership

Services

`service`	Start/stop/restart/enable services
`systemd`	Manage systemd units

Users & Groups

`user`	Create/modify/delete user accounts
`group`	Manage system groups

System

`command`	Run commands (no shell features)
`shell`	Run shell commands (pipes/redirects)
`cron`	Manage cron jobs

Network / Cloud

`uri`	Make HTTP requests
`get_url`	Download files from URL
`firewalld / ufw`	Manage firewall rules

Variables, Facts & Templates

Variables

```
# In playbook
vars:
  app_port: 8080
  db_host: localhost

# In task
- debug:
    msg: "Port is {{ app_port }}"

# Extra vars (CLI)
ansible-playbook site.yml \
  -e "app_port=9090"
```

Ansible Facts

```
# Auto-gathered system info
# Use in tasks/templates:

{{ ansible_hostname }}
{{ ansible_os_family }}
{{ ansible_distribution }}
{{ ansible_default_ipv4.address }}
{{ ansible_memtotal_mb }}

# Gather manually:
ansible all -m setup
```

Jinja2 Template

```
# nginx.conf.j2 (template)
server {
  listen {{ http_port }};
  server_name {{ ansible_hostname }};
  root /var/www/html;
}

# In playbook task:
- template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
```

Handlers & Conditional Tasks

Handlers — notify: on change

```
tasks:
  - name: Update nginx config
    template:
      src: nginx.conf.j2
      dest: /etc/nginx/nginx.conf
    notify: Restart Nginx

  - name: Another task
    copy:
      src: index.html
      dest: /var/www/html/
    notify: Restart Nginx

handlers:
  - name: Restart Nginx
    service:
      name: nginx
      state: restarted
# Only runs ONCE even if notified multiple times!
```

Conditionals — when:

```
tasks:
  - name: Install on Debian/Ubuntu
    apt:
      name: nginx
      state: present
    when: ansible_os_family == "Debian"

  - name: Install on RedHat
    yum:
      name: nginx
      state: present
    when: ansible_os_family == "RedHat"

  - name: Run only on Mint
    debug:
      msg: "This is Linux Mint!"
    when: ansible_distribution == "Linuxmint"
```

Loops & Tags in Playbooks

Loops — loop: / with_items:

```
# Install multiple packages
- name: Install packages
  apt:
    name: "{{ item }}"
    state: present
  loop:
    - nginx
    - git
    - curl
    - vim

# Loop with dict
- name: Create users
  user:
    name: "{{ item.name }}"
    groups: "{{ item.groups }}"
  loop:
    - { name: 'alice', groups: 'sudo' }
    - { name: 'bob', groups: 'users' }
```

Tags — run specific parts only

```
tasks:
  - name: Install nginx
    apt: name=nginx state=present
    tags:
      - install
      - web

  - name: Configure nginx
    template:
      src: nginx.conf.j2
      dest: /etc/nginx/nginx.conf
    tags: config

  - name: Start nginx
    service: name=nginx state=started
    tags: service

# Run only 'install' tagged tasks:
ansible-playbook site.yml --tags install
# Skip 'config' tasks:
ansible-playbook site.yml --skip-tags config
```

Ansible Roles — Organized Automation

Role Directory Structure

```
roles/
  webserver/
    tasks/
      main.yml       # Main task file
    handlers/
      main.yml       # Handlers
    templates/
      nginx.conf.j2 # Jinja2 templates
    files/
      index.html     # Static files
    vars/
      main.yml       # Variables
    defaults/
      main.yml       # Default variables
    meta/
      main.yml       # Role metadata
    README.md
```

Using Roles in a Playbook

```yaml
---
- name: Configure web servers
  hosts: webservers
  become: yes
  roles:
    - webserver
    - php
    - mysql
```

Create Role with ansible-galaxy

```
# Create role skeleton
ansible-galaxy init roles/webserver

# Install community role
ansible-galaxy install geerlingguy.nginx
```

Hands-On Lab Exercises

LAB 1: Setup & Ping

- Install Ansible on Linux Mint or AlmaLinux
- Create inventory file with localhost
- Run: ansible all -m ping
- View ansible --version output

LAB 2: Ad-Hoc Practice

- Check disk space on all hosts
- Install 'tree' package via ad-hoc
- Create a directory /tmp/ansible-test
- Gather facts about managed node

LAB 3: First Playbook

- Write a playbook to install nginx
- Add a task to start nginx service
- Add a task to copy index.html
- Run: ansible-playbook site.yml

LAB 4: Variables & Templates

- Define vars: port, app_name
- Create a Jinja2 template config file
- Deploy template to managed node
- Verify rendered file on remote host

Ansible Quick Reference Cheatsheet

```
ansible all -m ping
```
Test connectivity to all hosts

```
ansible all -m setup
```
Gather facts from all hosts

```
ansible-playbook site.yml
```
Run a playbook

```
ansible-playbook site.yml --check
```
Dry run (no changes)

```
ansible-playbook site.yml --diff
```
Show config file diffs

```
ansible-playbook site.yml -v
```
Verbose output

```
ansible-playbook site.yml -vvv
```
Very verbose / debug mode

```
ansible-playbook site.yml -l
webservers
```
Limit to specific group

```
ansible-playbook site.yml --tags
install
```
Run only tagged tasks

```
ansible-vault encrypt vars.yml
```
Encrypt a file with Vault

```
ansible-vault decrypt vars.yml
```
Decrypt a vault file

```
ansible-galaxy init roles/myrole
```
Create new role structure

```
ansible-galaxy install
username.rolename
```
Install role from Galaxy

```
ansible-inventory --list
```
Show inventory as JSON

```
ansible-doc -l
```
List all available modules

```
ansible-doc yum
```
Show docs for 'yum' module

Key Takeaways

- Ansible is agentless — uses SSH, no software on managed nodes
- Idempotency means running tasks multiple times is safe
- Playbooks are YAML files — readable, reusable automation scripts
- Modules are the building blocks — 2000+ available
- Ad-Hoc commands are quick one-liners for immediate tasks
- Roles organize complex automations for reuse & sharing
- Inventory defines your hosts — static or dynamic
- Vault secures sensitive data like passwords and API keys

Thank You! 🎉 Happy Automating!

Ansible Inventory — What & Why

NEW

What is an Inventory?

An Inventory is a file (or script) that tells Ansible WHICH machines to manage. It lists hostnames or IP addresses, organises them into groups, and can store per-host variables.

Define Hosts

List every server Ansible should manage — by IP or hostname

Group Servers

Organise into logical groups: webservers, dbservers, staging, prod

Per-Host Vars

Set host-specific variables like ansible_user, ansible_port, ansible_ssh_pass

Dynamic Hosts

Auto-discover hosts from AWS/Azure/GCP via dynamic inventory scripts

Default location: /etc/ansible/hosts | Override with: -i /path/to/inventory or in ansible.cfg

Inventory File Formats

NEW

Format 1 — INI (most common)

```
# /etc/ansible/hosts  (INI format)

# Ungrouped hosts
192.168.1.5
server1.example.com

# Web servers group
[webservers]
192.168.1.10
192.168.1.11  ansible_user=admin

# Database servers group
[dbservers]
db01.example.com  ansible_port=22
db02.example.com

# Group of groups
[production:children]
webservers
dbservers

# Group variables
[webservers:vars]
ansible_user=ubuntu
http_port=80
```

Format 2 — YAML (modern, flexible)

```
# inventory.yml  (YAML format)

all:
  hosts:
    192.168.1.5:
  children:
    webservers:
      hosts:
        192.168.1.10:
          ansible_user: ubuntu
        192.168.1.11:
          ansible_user: admin
          http_port: 80
    dbservers:
      hosts:
        db01.example.com:
          ansible_port: 22
        db02.example.com:
      vars:
        ansible_user: dbadmin
    production:
      children:
        webservers:
        dbservers:
```

ansible.cfg — When to Use & Root Privilege

NEW

When to Use ansible.cfg?

- ✓ You want to set a default inventory so you don't type -i every time
- ✓ You need to disable host key checking for new/test servers
- ✓ You want to set a default remote user for all connections
- ✓ You need to enable logging to a file for auditing
- ✓ You want to tune performance (forks = parallel tasks at once)
- ✓ You need privilege escalation (become: sudo) on by default

ansible.cfg — Root Privilege Settings

```
[defaults]
inventory      = /etc/ansible/hosts
remote_user    = ansible
host_key_checking = False

[privilege_escalation]
become         = True
become_method  = sudo
become_user    = root
become_ask_pass = False
# True = prompt for sudo password
# False = passwordless sudo required
```

Understanding Root Privilege Options

become: True
Enable privilege escalation globally

become_method: sudo
Use sudo to escalate (default)

become_user: root
Escalate to root user

become_ask_pass: False
Don't prompt — passwordless sudo

become_ask_pass: True
Prompt for sudo password at runtime

Ad-Hoc Commands — Root & Passwords

NEW

Important Flags for Root & Password

`-b or --become`	Activate privilege escalation (run as root via sudo)
`--become-user=root`	Specify WHICH user to become (default is root)
`-K or --ask-become-pass`	Prompt for the SUDO password at runtime
`-k or --ask-pass`	Prompt for the SSH login password
`--become-method=sudo`	Escalation method: sudo / su / pbrun / pfexec

Ad-Hoc Command Examples with Root / Password

Passwordless sudo (pre-configured):

```
ansible all -m yum -a 'name=nginx state=present' -b
```

Prompt for sudo password (-K):

```
ansible all -m yum -a 'name=nginx state=present' -b -K
```

SSH password + sudo password:

```
ansible all -m ping -k -K
```

Become specific user + sudo pass:

```
ansible all -m shell -a 'whoami' -b --become-user=root -K
```

Pass sudo password via extra vars:

```
ansible all -m apt -a 'name=git state=present' -b -e 'ansible_become_pass=MyPass123'
```

⚠ Never hardcode passwords in playbooks. Use ansible-vault encrypt_string or environment variables for production!